

Goal

Phase 1. Building Strong Foundation

1. Large Language Models (LLMs) Concepts
2. Introduction to LLMs in Python
3. Transformer Models with PyTorch
4. Fine-Tuning with Llama 3
5. Large Language Models for Business

Phase 2. RAG and LLM Applications

1. LLM Application Fundamentals with LangChain
2. Developing LLM Applications with LangChain
3. Retrieval Augmented Generation (RAG) with LangChain
4. End-to-End RAG with Weaviate
5. Graph RAG with LangChain and Neo4j
6. Introduction to Generative AI in Snowflake
7. Working with the OpenAI Responses API

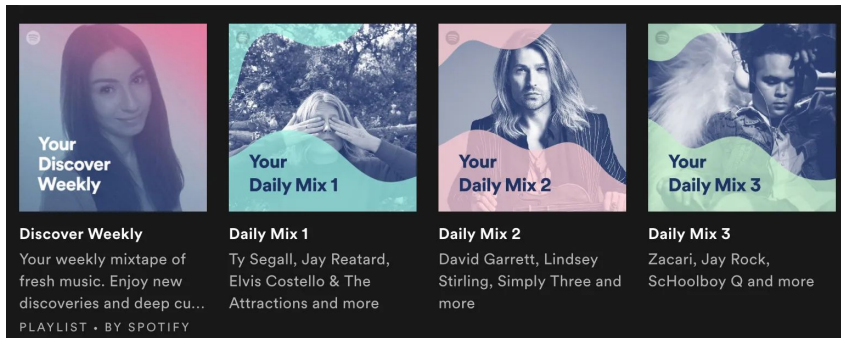
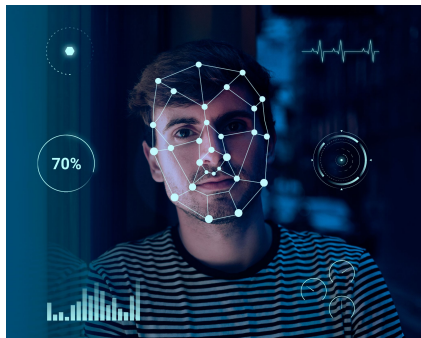
Phase 3. Agents, MCP, and Production AI

1. Introduction to AI Agents
2. Google: Agent Fundamentals
3. AI Agents with Hugging Face smolagents
4. LLM Tool Use with LangChain
5. Designing Agentic Systems with LangChain
6. Agentic Systems with LangGraph
7. Multi-Agent Systems with LangGraph
8. Building Agentic Workflows with LlamaIndex
9. Building AI Agents with Haystack
10. Text-to-Query Agents with MongoDB and LangGraph
11. Google: Manage Agent Memory and State
12. Building Scalable Agentic Systems
13. Introduction to Model Context Protocol (MCP)
14. LLM Ops Concepts
15. LLM Application Evaluation with LangSmith

Introduction to Large Language Model

LLM - Birds Eye View

Rapid Development in AI

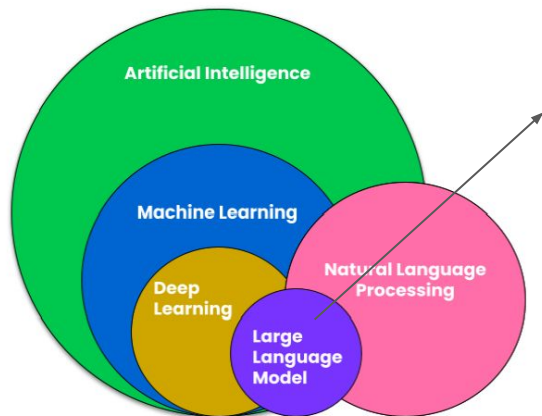


These are Great!! But...

AI traditionally lacked the ability to understand context, respond to open-ended questions, and generate human-like responses in conversation.

Introduction to Large Language Model

LLMs are the breakthrough



LLMs are based on DL architecture (transformers) those have capability of learning complex patterns using text.

LLMs are called **large** because they require a large amount of training data, computational resources to train. They have broad capabilities that comes with huge amount of trainable parameters.

LLM are perfect for use case which are **repetitive** and **impractical to scale**

Applications

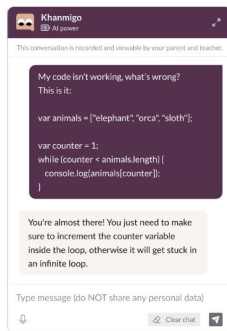
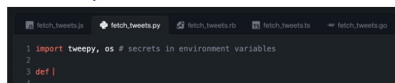
- Sentiment analysis - Analyze customer reviews and feedback
- Identifying themes & captioning
- Translating text or speech
- Summarizing & Generating text & code
- Next-word prediction
- Q&A
- Language translation
- Improve customer support with chatbots
- Increase employee productivity

- Generates trend forecasts and visual charts
- Efficiently transcribes virtual team meetings
- Assists in scriptwriting and revising.

- Enhance automation & Autonomous agents



Github copilot



- 🔍 attention is
- 🔍 attention is - Google Search
- 🔍 attention is all you need
- 🔍 attention is not explanation
- 🔍 attention is the new oil
- 🔍 attention is the purest form of generosity
- 🔍 attention is the new currency
- 🔍 attention is verb or noun
- 🔍 attention is drawn to meaning
- 🔍 attention is selective
- 🔍 attention is not love

Real World Application

LLMs are powerful AI tools that help businesses across industries by **automating** tasks, improving **efficiency**, and creating new **opportunities & revenue**

Finance: They can read and analyze unstructured text like reports, news, and social media to spot **market trends**, **manage investments**, and find opportunities.

Healthcare: Doctors' notes and medical records are often complex. LLMs can process this data to give personalized treatment recommendations, but privacy laws must be followed.

Education: LLMs act like AI tutors, answering questions, adapting teaching styles, and generating customized learning materials for each student

Customer Support Chatbot: understand sentiment analysis with irony detection & sarcasm
Content Creation

Multimodal AI: Unlike text-only models, multimodal LLMs handle multiple data types (text, audio, video, images).

NEWS

Microsoft launches AI-powered Security Copilot

Microsoft Security Copilot is an AI assistant for infosec professionals that combines OpenAI's GPT-4 technology with the software giant's own cybersecurity-trained model.



TECH

Bloomberg plans to integrate GPT-style A.I. into its terminal

Stripe and OpenAI collaborate to monetise ChatGPT and other technologies

Snapchat released its AI persona, My AI, that allows users to prompt it for recipe suggestions, gift ideas, and content inspiration. It runs off of OpenAI's ChatGPT.

**BloombergGPT:
A Large Language
Model for Finance**

Bloomberg

Model aims to improve financial tasks such as:

- Sentiment analysis
- Entity recognition
- News classification

Challenges of Language Modeling

LLMs have to understand language like humans do, which is difficult because:

Word Order Matters (Sequence)

The position of a word can change the meaning.

- "I only follow a healthy lifestyle." → I do nothing else.
- "Only I follow a healthy lifestyle." → No one else does.

A model must understand the order of words correctly.

Words Depend on Context

The same word can have different meanings.

- "I run a marathon." → jog
- "I run an organization." → manage
- "I run a machine." → operate

The model uses surrounding words to determine the correct meaning.

Long-Range Dependencies

Sometimes related words are far apart in a sentence.

"The book that the young girl placed on the shelf was quite heavy."

The model must connect **"book"** with **"heavy"**, even though many words separate them.

Single Task Learning

Traditional models were trained for only one task.

- One model for translation
- Another for summarization
- Another for question answering

This required more time, data, and resources.

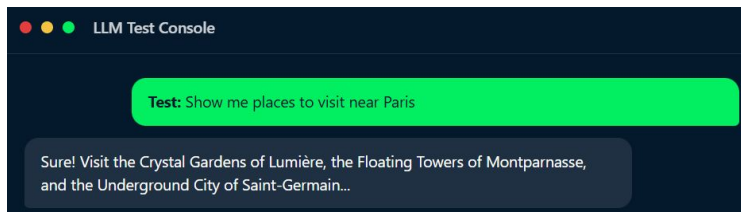
Multi-Task Learning (Modern LLMs)

Modern LLMs can perform many tasks using the same model.

Examples:

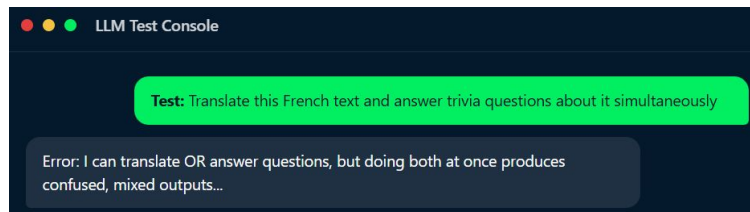
- Answer questions
- Summarize text
- Translate languages
- Write code

Challenges of Language Modeling



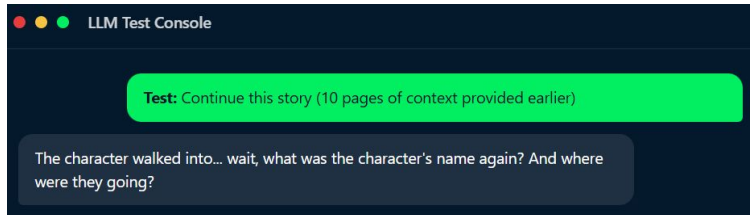
Contextual Awareness

The model generates fictional places that don't exist. This is a contextual awareness challenge — the model lacks grounding in real-world facts and can confidently produce incorrect information (hallucination).



Multi-Task Learning

The model struggles to handle translation and trivia simultaneously. This is a multi-task learning challenge — performing multiple distinct tasks at once degrades quality on each.



Sequential Pattern Recognition

The model forgets key details from earlier in a long input. This is a sequential pattern recognition challenge — models can lose track of context over long sequences

Novelty of LLM

LLMs can understand subtle meanings in language, such as Sarcasm, Humor , Irony, Intent, Tone.

Text: "Oh great, another meeting."

Traditional model: "What's the meeting about?"

LLM: "Sounds like you're really excited about it!"

(understands sarcasm)

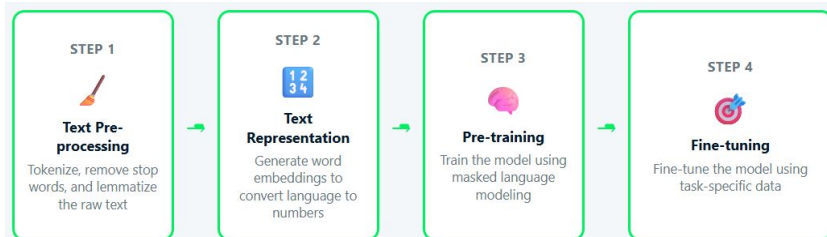
How does it happen ?

- **Huge amounts of training data**
- **Billions of parameters**
Parameters are like the model's knowledge and learned patterns.
- **Emergent Abilities:** When a model becomes large enough, new abilities suddenly appear.
 - Writing poetry
 - Generating code
 - Answering complex questions
 - Assisting with medical knowledge
 - Holding natural conversations

These abilities are called **emergent capabilities**.

How LLMs Are Trained

1. Text processing
 - Tokenization
 - Lemmatization
 - Stop word removal
2. Text Representation
 - Bag of words
 - Word embeddings
3. Pre-training: Model learns general language patterns from huge amounts of text.
4. Fine tuning
 - Zero shot learning
 - Few shot learnings
 - Multi shot learning



NLP Preprocessing & Text Representation

Tokenization: Split a sentence into individual words (tokens).

Sentence: Working with NLP is tricky.

Tokens: ["Working", "with", "NLP", "is", "tricky", "."]

Stop Word Removal: Remove common words that add little meaning.

Before: ["Working", "with", "NLP", "is", "tricky"]

After: ["Working", "NLP", "tricky"]

Removed words: **with, is**

Lemmatization

Convert words to their base (root) form.

talking → talk

talked → talk

talks → talk

This helps treat similar words as the same word.

Text Representation

Computers understand numbers, not words.

So we convert text into numbers.

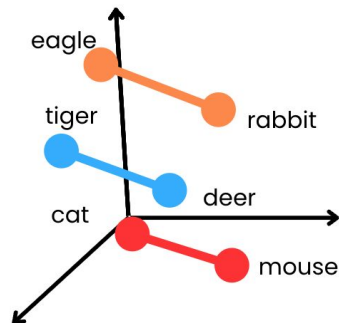
Bag of Words (BoW)

Count how many times each word appears.

S1: The cat chased the mouse swiftly

S2: The mouse chased the cat

Word	S1	S2
cat	1	1
chased	1	1
mouse	1	1
swiftly	1	0



Word Embeddings

Convert each word into a list of numbers that captures its meaning.

cat → [-0.9, 0.9, 0.8]

tiger → [-0.8, 0.85, 0.9]

Since **cat** and **tiger** are similar animals, their numbers are similar.

Generative Pretraining

Before an LLM can answer questions or write text, it is **pre-trained** on huge amounts of text.

Two common pre-training methods

Next Word Prediction

The model learns by predicting the next word in a sentence.

Training data for next word prediction

Input	Output
The quick brown	fox
The quick brown fox	jumps
The quick brown fox jumps	over
The quick brown fox jumps over	the
The quick brown fox jumps over the	lazy
The quick brown fox jumps over the lazy	dog
The quick brown fox jumps over the lazy dog.	

By seeing millions of such examples, the model learns which words usually come next.

Prompt: I love pizza with ____

The model is likely to predict: cheese

because it has seen "pizza with cheese" many times in its training data

Purpose: Teach the model how to generate natural text.

Masked Language Modeling (MLM)

The model learns by filling in missing words.

Example: The quick brown fox jumps over the lazy dog.

Masked: The quick **[MASK]** fox jumps over the lazy dog. The model must predict: brown

It uses the surrounding words ("quick" and "fox") to guess the missing word.

Purpose: Teach the model to understand context and relationships between words.

Transformer - Attention Is All You Need

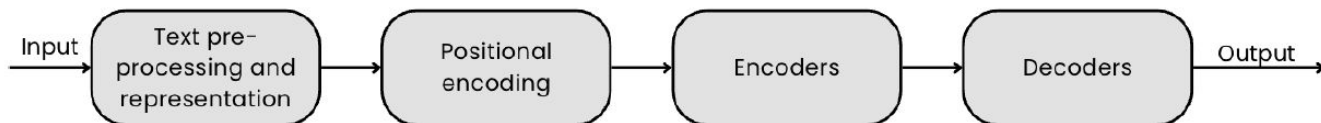
Transformers can process many words **at the same time (in parallel)**, making them faster and better at handling long text.

It was introduced in the famous paper: "**Attention Is All You Need**"

Transformers help models understand language better by focusing on **important words** and **relationships** in a sentence.

Components of a Transformer

1. **Text Preprocessing:** The sentence is broken into tokens (words/subwords) and converted into numbers.
2. **Text Representation:** The sentences are converted into vectors
3. **Positional Encoding:** The model is told the position of each word.
This helps the model understand word order.
4. **Encoder:** The encoder reads and understands the input sentence.
5. It uses an **attention mechanism** to find important relationships between words.
Example "Jane, who lives in New York, loves restaurants."
Even though these words are far apart the encoder learns that:
 - "Jane" is the person
 - "loves" is Jane's action
6. **Decoder:** The decoder generates the output one word at a time.



Types of Transformer Architecture

Encoder-Only

Purpose: Used for text classification, sentiment analysis,
Extractive question answering

Example: BERT

Think of it as: Reading and understanding a document, but not writing new text.

Decoder-Only

Purpose: Generate text

Example: Text generation, Chatbots, Generative question answering

Think of it as: Continuing or creating text based on a prompt.

Input: "Once upon a time..."

Output: A generated story

Encoder-Decoder

Purpose: Understand input and generate output

Used for: Translation, Summarization, Text-to-text tasks

Example Models: T5, BART

Think of it as: Reading one piece of text and then rewriting it.

Input: Spanish sentence

Output: English translation

Transformer - Attention Is All You Need

Why Are Transformers Special

- 1. Handle Long-Range Dependencies:** They can connect words that are far apart.
- 2. Use Attention:** The model focuses on the most relevant words instead of treating all words equally. This improves understanding and text generation.
- 3. Process Many Words at Once**

Attention Mechanism

Attention helps an LLM focus on the most important words in a sentence, instead of treating all words equally.

Self-Attention

Self-attention means the model looks at **all words in a sentence and figures out how each word relates to every other word**

The boy went to the store because he wanted groceries.

The model understands:

- “he” → refers to “boy”
- “groceries” → related to “store”

It helps the model understand connections inside one sentence.

Multi-Head Attention

Multi-head attention means the model uses **multiple “attention views” at the same time** to process multiple aspects of text simultaneously

Each head focuses on a different thing:

- One head → main subject (boy)
- Another head → actions (went, bought)
- Another head → objects (store, groceries, cereal)

Then all results are combined for a better understanding

Fine Tuning

A pre-trained LLM already knows general language, grammar, facts, and reasoning because it has been trained on a huge amount of text.

Fine-tuning means taking an already trained LLM and teaching it a specific task or domain.

Customer support bot → Fine-tune on past customer conversations.

Financial assistant → Fine-tune on financial reports and market data.

Sentiment analysis → Fine-tune on labeled examples of positive, negative, and neutral text.

Why Not Train From Scratch ?

- Huge Computing Power
- Long Training Time
- Massive Amounts of Data

Fine Tuning Support in Hugging Face

Pipeline

- Easy to use.
- Automatically loads the model and tokenizer.
- Good for inference (making predictions).
- Less flexibility.
- `pipeline("sentiment-analysis")`

Auto Classes

- More control and customization.
- Used when fine-tuning models.

Fine Tuning Techniques

1. Full Fine-Tuning (Classic approach) : You update **all model weights** on your task dataset.
How it works: Start with a pre-trained model and continue training on labeled data.
Pros: Best performance when you have lots of data and compute.
Cons: Very expensive (memory + GPU), and risk of “forgetting” general knowledge.
2. Feature-based Fine-Tuning (Frozen backbone): You freeze the base model and train only a small task-specific head. Example: Add a classification layer on top of embeddings.
Common in older NLP pipelines.
Fast, but not very powerful for generative LLM tasks
3. **Instruction Fine-Tuning (Supervised Fine-Tuning / SFT):** This is one of the most important modern methods
Train on (**instruction** → **response**) datasets.
Example:
 - Input: “Summarize this paragraph”
 - Output: “Summary...”This is what makes models like chat assistants actually follow instructions. Often done using tools like TRL’s **SFTTrainer**.
 - **Pros:** Great for chat behavior and task following
 - **Cons:** Needs high-quality instruction data

Fine Tuning Techniques

Parameter-Efficient Fine-Tuning (PEFT)

Instead of updating all parameters, you train only a **small subset of additional parameters**.

4.1 LoRA (Low-Rank Adaptation)

- One of the most widely used methods.
- Instead of updating weight matrix W , you add low-rank matrices A and B :
 - $W' = W + A \cdot B$
- Base model is frozen; only A and B are trained.

Pros: Very memory efficient

Cons: Slight performance trade-off vs full fine-tuning

4.2 QLoRA: Extension of LoRA + quantization.

- Base model is loaded in **4-bit precision**
- LoRA adapters are trained on top

Pros: Fine-tune huge models on a single GPU

4.3 Adapters

Small neural modules inserted between layers.

- You freeze the model and insert trainable “adapter blocks”
- Each task can have its own adapters

4.4 Prompt Tuning / Soft Prompting

Instead of changing model weights:

- You train a small set of **learnable prompt embeddings**
- These are prepended to input tokens

Think of it as learning “best possible prompt vectors.”

4.5 Prefix Tuning

Similar to prompt tuning, but instead of input embeddings:

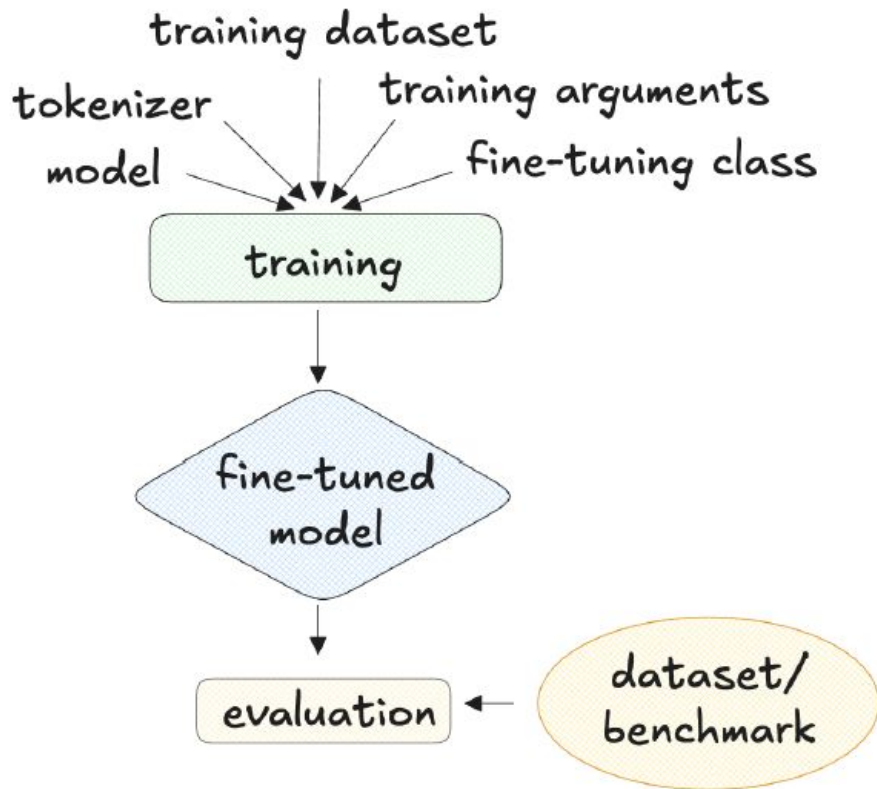
You inject trainable vectors into **each transformer layer’s attention keys/values**

More powerful than prompt tuning, but slightly more complex.

Fine Tuning

What do we need

- **Pre-trained Llama model** – the model we start with.
- **Tokenizer** – converts text into numbers and back.
- **Training dataset** – examples of customer questions and answers.
- **Training Arguments** – settings that control how learning happens.
- **SFTTrainer** – the tool that performs the fine-tuning.



Fine Tuning - Quantization

Quantization: Conversion from higher memory format to lower memory format

Weights are stored in 32 bits FP → 8 bits HP

Loss of info

It is very helpful in inference

How to Perform Quantization

1. Symmetric unsigned int8 Quantization: FP 16 Sign exponent mantissa
 - a. Calibration
2. Asymmetric Quantization

Post training Quantization

Pre trained model → Calibration → Quantized model → use cases

Loss of data

Quantization Aware Training (QAT)

Pre trained model → Calibration → Fine Tuning → Quantized model

Training Data

Fine Tuning - Low Rank Adaption of LLM

Full Fine Tuning: Update all model weights

LORA

Instead of Updating weights, it track changes

Pre Trained Weights + LORA Tracked weights = Fine Tuned Weights

LORA Tracked weights is decomposed to smaller matrix $\rightarrow 3 \times 3 = 3 \times 1 \times 3$

Quantized LORA (LORA)

PEFT: Parameter Efficient Fine Tuning

Freeze weights of base model

Fine Tuning

TorchTune

- Based on configurable templates
- Ideal for: scaling quickly

SFTTrainer (Supervised Fine-Tuning) from Hugging Face

- Access to other LLMs
- Ideal for: fine-tuning multiple models

Unsloth

- Efficient memory usage
- Ideal for: limited hardware

Axolotl

- Modular approach
- Ideal for: no extensive reconfiguration

Fine Tuning Llama with Torch Tune

TorchTune provides ready-made recipes for:

- Full Fine-Tuning: update all the weights
- LoRA Fine-Tuning (recommended)
- QLoRA Fine-Tuning
- Distributed Training
- Instruction Tuning
- Continued Pretraining

Fine Tuning Techniques

N-Shot Learning

Zero-Shot Learning

The model performs a task **without seeing any examples**.

Prompt: Classify this review as Positive or Negative:

"The movie was amazing!"

Even if the model was never specifically trained for this task, it can still answer: **Positive**

One-Shot Learning

The model learns from **one example**.

Example: "I love this product" → Positive

Now classify: "This phone is great"

Answer: Positive

Few-Shot Learning

The model learns from **a few examples (2–10 typically)**.

Example: "Amazing service" → Positive

"Terrible experience" → Negative

"Very helpful staff" → Positive

Multi-Shot Learning

The model learns from **many examples**, but still fewer than full-scale training.

The more examples it sees, the better it usually performs.

Example: Showing dozens or hundreds of customer reviews before asking it to classify a new review.

Metrics



**Text
classification**

Accuracy

F1 score



**Text
generation**

Perplexity

BLEU score



Summarization

ROUGE score

BLEU score



Translation

BLEU score

METEOR



**Question-
answering**

Extractive QA

Exact Match (EM)
F1 score

Generative QA

BLEU / ROUGE

Metrics for Text Generation Task

Perplexity : Measures how **confident the model is when predicting the next word**.

It checks how “surprised” the model is by the text.
Low perplexity = good model (more confident, better predictions). High perplexity = poor model (confused predictions)

BLEU Score : Compares model-generated text with **human-written reference text**.

Used for: Translation, Summarization, Text generation evaluation

Interpretation: Score ranges from **0 to 1**. 0 = completely different and 1 = perfect match

ROUGE Score: ROUGE evaluates how much of the reference text is captured by the generated text.
Common for Summarization

METEOR Score: Improves upon BLEU by considering:
Exact word matches Stemming, Synonyms, Word order
Reference: The boy is running.
Prediction: The child is jogging.
BLEU may score low. METEOR recognizes: boy $\approx$ child.
running $\approx$ jogging and gives a better score.
Range: 0 to 1

Exact Match (EM) : Checks whether prediction exactly equals reference.
Very common in Question Answering.

Toxicity Metric: Measures whether generated text contains: Hate speech, Abuse, Insult, Offensive language, Harassment .
0 $\rightarrow$ safe
1 $\rightarrow$ highly toxic

Reinforcement Learning from Human Feedback

RLHF is a way to make LLMs better using human guidance.

It is the **final improvement step** after:

- Pre-training (learn general language)
- Fine-tuning (learn specific tasks)

Why do we need RLHF ?

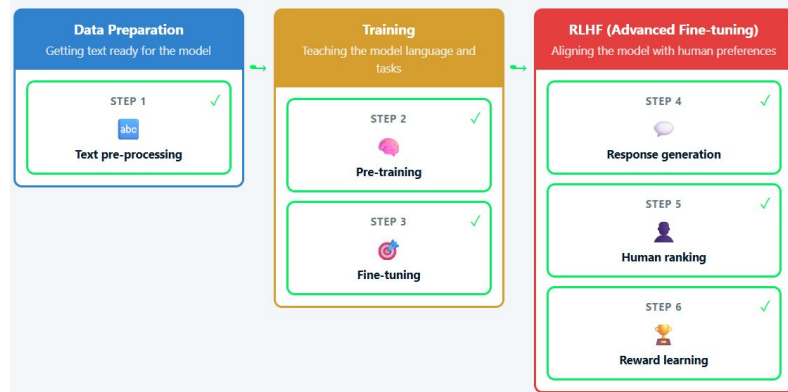
Because training data from the internet can:

- Contain wrong information
- Have noise or bias
- Include mixed opinions

So the model may sometimes give **unreliable answers**. RLHF fixes this using human judgment.

How RLHF works

- The AI gives multiple possible responses to a question.
- A human expert:
 - Checks the answers
 - Ranks them (best → worst)
 - Chooses which is most correct and helpful
- The model learns from this feedback



Data concerns and considerations

5 Important things to consider while building LLMs

1. Data Volume & Computing Power

- LLMs need **huge amounts of data** to learn language patterns. Example: Some models are trained on data equal to **more than 1 million books**.
- Processing such large datasets requires **powerful computers** and can cost **millions of dollars**.

2. Data Quality

- The training data should be **accurate, clean, and well-written**.
- If the data contains mistakes, poor grammar, or incorrect information, the model will learn those mistakes.

3. Correct Labels

- Some training tasks need data to be labeled correctly. Example: News articles may be labeled as **Sports**, **Politics**, or **Technology**.
- Wrong labels confuse the model and reduce its accuracy. Creating and checking labels often requires a lot of human effort.

4. Data Bias

- Training data may contain stereotypes or represent some groups more than others. This can cause unfair or biased responses. Example: The model might assume a nurse is female because that pattern appears often in the data.
- To reduce bias: Use diverse datasets. Check for imbalances. Apply bias-mitigation techniques.

5. Data Privacy

- Training data may contain sensitive personal information (PII). Using private data without permission can violate privacy laws.

Data concerns and considerations

Problems due to data quality issues and biases

Data Bias

1. The model trained mostly on action movie data, while the test data consists of romantic movies.
2. A recommendation system for a music streaming service, trained primarily on pop music, fails to recommend country music to a country music fan.
3. An AI model for recognizing animal species keeps misidentifying certain bird species as its training data included fewer examples of those birds.

Data Quality and Labeling

1. A translation model trained on English texts produces spelling and grammatical errors.
2. A predictive model trained on sports data confuses american football with soccer.
3. A stock prediction model trained on fake stock price data makes inaccurate predictions on future stock prices.

Ethical and Environmental concerns

1. Ethical Concern

- **Transparency Risk**
 - LLMs are often like a **black box**. We can see the answer, but not always understand **how the model reached that answer**. This is a problem in important areas like healthcare, where doctors need explanations for decisions.
 - **Example:** If an AI predicts a patient has a disease, it should explain why.
- **Accountability Risk**
 - If an AI gives wrong or harmful advice, who is responsible? The AI developer? & The company using the AI?
 - It can be difficult to assign responsibility.
 - **Example:** An AI gives incorrect medical advice that harms a patient.
- **Information Hazards:** These are risks from harmful information generated by AI.
 1. **Harmful Content:** AI may generate offensive, inappropriate, or disturbing content.
 2. **Misinformation:** AI can sound confident even when it is wrong. It may provide false information or unproven advice.
 3. **Malicious Use:** People may use AI to create fake news, scams, or misleading content.
 4. **Toxicity:** AI may generate biased, discriminatory, or offensive responses.
 - Exposure to diverse and representative training data
 - Bias audits on model outputs + bias removal techniques
 - Fine-tune to specific use cases in sensitive applications
 - Prompt engineering: carefully crafting and refining prompts

2. Environmental Concerns

High Energy Consumption: Training LLMs requires thousands of powerful computers. These computers consume a

Flexibility and its consequences

Same prompt -> different responses

Marketing team creates product description using LLMs:

- Customers see different descriptions
- Brand image could be damaged

Security alert!

Attackers can embed harmful prompts on websites

- Can distort LLMs outputs (**prompt injection**)

Where are LLMs heading?

1. Better Explainability

- Future LLMs should explain **why** they gave a particular answer.
- This helps users trust the model and identify mistakes or bias.
- **Example:** If an AI suggests a travel route, it should explain why it chose that route.

2. Higher Efficiency

- Current LLMs need a lot of computing power and energy.
- Researchers are making models: Faster, Smaller, Less expensive, More energy-efficient
- **Benefits:** Lower costs, Less electricity usage, Can run on phones and smaller devices

3. Automatic Bias Detection

- Future LLMs may automatically find and reduce bias in their training data.
- This could happen without humans manually labeling data.
- **Challenge:** Some biases are subtle, so the AI might miss them or introduce new biases.

4. Enhanced Creativity

- LLMs can already create: Stories, Poems, Music, Art (with other AI models)
- Future research aims to make AI interactions feel more natural and emotionally aware.
- **Important:** AI does not truly understand emotions like humans; it only learns patterns from data.

